AVL Tree Dictionary Project Report

Amer Aziz Almutairi ID:
202169770
Date: May 03, 2023

Table of Contents

1. Abstract

This report presents the design and implementation of a personal dictionary application utilizing an AVL Tree for balanced storage and efficient operations. Core functionalities include insertion, deletion, lookup, and similar-word search based on single-character edits. Performance considerations, testing methodologies, and encountered challenges are discussed.

2. Introduction

Efficient data retrieval and dynamic set operations are fundamental to many software systems. To address these requirements, this project implements a dictionary data structure backed by a self-balancing AVL Tree. The AVL Tree ensures O(log n) performance for insertions, deletions, and lookups by maintaining strict height balance through rotations.

3. Motivation

- Provide a robust personal vocabulary management tool. - Leverage the balanced nature of AVL Trees for guaranteed operation times. - Gain hands-on experience implementing advanced tree structures.

4. Methodology

4.1 AVL Tree Implementation

The AVL Tree class extends a basic binary search tree with automatic balancing. Each node tracks its height, and rotations (single and double) are applied when the balance factor exceeds permitted thresholds.

4.2 Dictionary Operations

Operations are exposed through a command-line interface in Dictionary.java. Key methods: addWord(String s): Inserts a new entry, throwing an exception on duplicates. findWord(String s): Checks for existence in O(log n) time. - deleteWord(String s): Removes an entry, rebalancing as needed. - findSimilar(String s): Generates single-character variations and filters by existence.

5. Testing

Comprehensive unit tests cover edge cases including empty dictionaries, boundary values, duplicate insertions, and invalid deletions. Performance tests confirm balanced-tree properties across large datasets.

6. Challenges and Solutions

- Understanding AVL rotations: Consulted algorithmic references to ensure correctness. Handling edge conditions: Implemented thorough null and bounds checks. - Efficient file I/O: Adopted recursive traversal to serialize tree contents effectively.

7. Conclusion

The project demonstrates a full-featured dictionary application with guaranteed operational performance. Future work may include GUI integration, persistence optimizations, and support for multi-character edit searches.